



Programas para el curso

APUNTES TÉCNICOS

PARADIGMAS DE LA PROGRAMACIÓN

Para el apoyo del curso

Autor:
José Sánchez Juárez

Febrero 2026

Índice general

1. Capítulo 1	1
1.1. Introducción	1
1.1.1. Concepto 1	1
1.1.2. Concepto 2	3
1.1.3. Actividades	8
1.2. Continuación de paradigma	8
1.2.1. Declarativo	9
1.3. Continuación de paradigma	11
1.3.1. Listas	11
1.3.2. Funcional	12
1.4. Continuación de paradigma	24
1.4.1. Línea de comandos	24
1.4.2. Polimorfismo	26
1.5. Actividades	32
1.6. Lógico	32
2. Capítulo 2	33
2.1. Creación en C++	33
2.1.1. Clases y arreglos	35
2.1.2. Aplicación de static	36
2.2. Objetos	37
2.3. Herencia	38
2.4. Virtual	39
2.5. Modificadores	40
2.6. Interfaces	43
2.6.1. Uso de lambda	43
2.7. Java GUI	46
2.7.1. Listas	49

2.8. Paradigmas	51
2.9. Historia	51
2.10. Actividades	51
3. Capítulo 3	55
3.1. Proyecto 1	55
3.2. Proyecto 2	55
3.3. Proyecto 3	56
3.4. Proyecto 4	56
3.5. Proyecto 5	56
3.6. Proyecto 6	56
3.7. Proyecto 7	56
3.8. Proyecto 8	56
3.9. Proyecto 9	56
3.10. Proyecto 10	56
4. Capítulo 4	59
5. Capítulo 5	61
5.1. Tema	61

CAPÍTULO 1

Lenguajes y paradigmas de la programación

Las clases son los lunes, martes, y jueves a las 10:30, los martes a las 8:30. El curso se inicia el 3 de febrero del 2026 y termina el 1 de julio del 2026. Los días festivos son: lunes 16 de marzo; viernes 1, martes 5, viernes 15 de mayo. El receso de semana santa es: del lunes 30 de marzo hasta el viernes 10 de abril del 2026.

SECCIÓN 1.1

Clase 1

Se comienza con la dinámica, los alcances, la forma de evaluar el curso. Los estudiantes se registrarán en la plataforma **CLASSROOM** para compiladores. Se propone para la evaluación la siguiente ponderación: trabajo individual el 50 %; el trabajo colectivo el 40 %; la excelencia el 10 % que se evaluara por medio de un examen escrito el martes 23 de junio del 2026. El registro de calificaciones finales en el SAES es del miércoles 24 de junio al viernes 26 de junio del 2026.

1.1.1. Concepto de lenguaje

La comunicación entre personas se realiza por medio del lenguaje, lo mismo la comunicación entre el hombre y la computadora, también se realiza por medio del lenguaje. En el caso de la computación el lenguaje de alto nivel es el

que se aplica para realizar la actividad de comunicación con la computadora. La lamina ***PaPD1Lenguaje.dia*** expresa el concepto del lenguaje.

Ejemplos de lenguaje

Los lenguajes de programación deben ser concisos, breves y claros.

```
1 public class Hola {  
2     public static void main(String[] args) {  
3         System.out.println("Veo al mundo");  
4     }  
5 }
```

Listing 1.1: Hola en Java

```
1 print("Hola que bonito es el mundo")
```

Listing 1.2: Hola en Python

```
1 #include<iostream>  
2  
3 using namespace std;  
4  
5 int main() {  
6     cout << "Hola que bonito es el mundo";  
7 }
```

Listing 1.3: Hola en C++

El uso del teclado para alimentar datos:

```
1 import java.util.Scanner;  
2  
3 public class Entrada {  
4     public static void main(String[] args) {  
5         Scanner cadena = new Scanner(System.in);  
6  
7         String n = cadena.nextLine();  
8  
9         System.out.println(n);  
10    }  
11 }
```

Listing 1.4: Uso del teclado en Java

```
1 s = input("Hola que bonito es el mundo")  
2
```

```
3 print(s)
```

Listing 1.5: Uso del teclado en Python

```
1 #include<iostream>
2
3 using namespace std;
4
5 int main() {
6     string n;
7
8     cin >> n;
9
10    cout << n << endl;
11 }
```

Listing 1.6: Uso del teclado en C++

1.1.2. Concepto de paradigma

Paradigma expresa en computación una forma de abordar los problemas para realizar las instrucciones que se le aplican a la computadora por medio de un lenguaje de alto nivel. Se presenta la lamina ***PaPD2Paradigma.dia*** que expresa el concepto de paradigmas, donde se plantea la manera de resolver un problema donde se aplican los conceptos matemáticos.

Ejemplos de paradigmas

Para escribir código en diferentes paradigmas no depende del lenguaje. En un mismo lenguaje se puede escribir código en diferentes paradigmas. Los paradigmas que se presentaran en el curso, son: **imperativo, declarativo, funcional, y orientado a objetos.**

Paradigma imperativo Este paradigma consiste en decirle la manera de cumplir con las instrucciones dadas a la computadora. El programa consiste en una secuencia de sentencias que modifican el estado del programa. Donde el desarrollador especifica de manera detallada de como se deben realizar las tareas. El programador define una secuencia de comandos o pasos, y el flujo del programa es determinado por las estructuras de control tales como ciclos y condicionales.

Por ejemplo: decirle a la computadora que encuentre el último dígito de una cadena.

```
1 #include<iostream>
2
3 using namespace std;
4
5 int main() {
6     int n;
7
8     cout << "Escribe un numero entero de varios digitos.";
9     cin >> n;
10    cout << "su ultimo digito, es:";
11 }
```

Listing 1.7: Entregar el último dígito de un número en C++ sin decirle como.

```
1 #include<iostream>
2
3 using namespace std;
4
5 int main() {
6     int n;
7     int u;
8
9     cout << "Escribe un numero entero de varios digitos." <<
endl;
10    cin >> n;
11
12    u = n % 10;
13    cout << "el ultimo digito, es:" << endl;
14    cout << u << endl;
15 }
```

Listing 1.8: Entregar el último dígito de un número en C++ diciendole como.

```
1 import java.util.Scanner;
2
3 public class Encontrar1 {
4     public static void main(String[] args) {
5         Scanner num = new Scanner(System.in);
6
7         System.out.println("Escriba un numero de varios
digitos");
8
9         int dig = num.nextInt();
```

```
10     int u = dig % 10;
11
12     System.out.println("El ultimo digito, es: ");
13     System.out.println(u);
14 }
15 }
```

Listing 1.9: Entrega el último dígito de un número de varias cifras en Java.

```
1 n = int(input("Escribe un numero de varias cifras: "))
2
3 u = n % 10
4
5 print("El ultimo digito, es: ", u)
```

Listing 1.10: Entrega el último dígito de un número de varias cifras en Python

Otro ejemplo: calcular la suma de los números del 1 al 10.

```
1 t = 0
2 for i in range(1,11):
3     t += i
4 print(t)
```

Listing 1.11: Suma de números desde el 1 al 10 en Python

Otro ejemplo: determinar si un número es mayor o menor.

```
1 x = 10
2 if x > 5:
3     print("x es mas grande que 5")
4 else:
5     print("x es menor o igual que 5")
```

Listing 1.12: Determina si un número es mayor o menor en Python

Otro ejemplo: sumar todos los elementos de un arreglo.

```
1 num = [1.2.3.4.5]
2 t = 0
3 for num in num:
4     t += num
5 print(t)
```

Listing 1.13: Suma de todos los elementos de un arreglo en Python

```
1 #include<iostream>
2
3 using namespace std;
```

```
4
5 int main() {
6     int num[] = [1,2,3,4,5];
7     int t = 0,
8
9     for(int i = 0; i < 5; i++) {
10         t += num[i];
11     }
12
13     cout << t;
14 }
```

Listing 1.14: Suma de todos los elementos de un arreglo en C++.

Un ejemplo combinado del paradigma imperativo y el paradigma orientado a objetos, es el siguiente:

```
1 class Calculadora:
2     def inicializar(self):
3         self.total = 0
4
5     def add(self, num):
6         self.total += num
7
8 cal = Calculadora()
9 cal.add(10)
10 cal.add(5)
11 print(cal.total)
```

Listing 1.15: Combinación de paradigmas imperativo y orientado a objetos en Python

El uso de los ciclos:

```
1 for i in range(1, 6):
2     print(i)
3
4 contador = 1
5 while contador <= 5:
6     print(contador)
7     contador += 1
```

Listing 1.16: Uso de ciclos para el paradigma imperativo en Python

El uso de llamadas a función:

```
1 def saludo(nombre):
2     print(f"Hola, {nombre}!")
3
```

```
4 saludo("Alicia")
```

Listing 1.17: Uso de llamadas a función para el paradigma imperativo en Python

Uso de sentencias condicionales:

```
1 tem = 30
2
3 if tem > 25:
4     print("Tiene calentura")
5 elif tem > 15:
6     print("Esta tibio")
7 else
8     print("Esta frio")
```

Listing 1.18: Uso de sentencias condicionales para el paradigma imperativo en Python

Uso de break:

```
1 for i in range(1, 6):
2     if i == 3:
3         break
4     print(i)
```

Listing 1.19: Uso de break para el paradigma imperativo en Python

Uso de continue:

```
1 for i in range(1, 6):
2     if i == 3:
3         continue
4     print(i)
```

Listing 1.20: Uso de continue para el paradigma imperativo en Python

[1, página 48]

Programa que muestra lo mutable y lo inmutable:

```
1 miLista = [1,2,3]
2 miLista[0] = 10
3
4 print(miLista)
5
6 miCadena = "mundo"
7 miCadena[0] = "H"
```

Listing 1.21: Mutable e inmutable para el paradigma imperativo en Python

[1, página 51]

1.1.3. Actividades de la clase 1

Las actividades podrán extenderse por varias clases más. Contestar las siguientes preguntas y programas:

1. ¿Como se define el lenguaje?
2. ¿Como se estudia el lenguaje?
3. ¿Que tipo de lenguaje usamos en computación?
4. ¿Cuales son las características del lenguaje que usamos en computación?
5. Definir un paradigma.
6. De ejemplos de paradigmas en computación.
7. ¿Que diferencia hay entre el paradigma y el código?
8. ¿Que es programar?
9. De ejemplos de la forma de programar.
10. Construir un programa que permita leer un número entero y que muestre la mitad de su último dígito.
11. Construir un programa que permita leer un número entero y que muestre los dos últimos dígitos.
12. Construir un programa que permita leer dos enteros y que muestre la suma del último dígito de cada uno de los números leídos.

SECCIÓN 1.2

Clase 2

Continuando con los paradigmas el siguiente es el paradigma declarativo, este consiste en decirle a la computadora que se quiere y quien se encarga de como hacerlo, es el compilador.

1.2.1. El paradigma declarativo

Un ejemplo de lenguaje que cumple con este paradigma, es el SQL. Ejemplos de código en paradigma declarativo:

```
SELECT *FROM empleado WHERE edad > 30;
```

```
<h1>Bienvenido al sitio web</h1>
```

```
<p>Esto es una simple página web</p>
```

Un ejemplo seria: sumar todos los números de un arreglo.

```
1 num = [1,2,3,4,5]
2 t = sum(num)
3 print(t)
```

Listing 1.22: Suma de todos los elementos de un arreglo en Python basado en el paradigma declarativo.

```
1 #include<iostream>
2
3 using namespace std;
4
5 int main(){
6     int num[] = [1,2,3,4,5];
7     int t = 0,
8
9     t = suma(num);
10
11     cout << t;
12 }
```

Listing 1.23: Suma de todos los elementos de un arreglo en C++ basado en el paradigma declarativo (a prueba).

```
1 #include<iostream>
2
3 using namespace std;
4
5 int main() {
6     int num[] = [1,2,3,4,5];
```

```
7   int t = 0,  
8  
9   for(int i = 0; i < 5; i++) {  
10      t += num[i];  
11  }  
12  
13  cout << t;  
14 }
```

Listing 1.24: Suma de todos los elementos de un arreglo en C++ basado en el paradigma imperativo.

Realizar las siguientes actividades:

1. Construir un programa que permita leer un numero entero y que entregue sus dos últimos dígitos.
2. Construir un programa que permita leer dos enteros y que sume sus dos últimos dígitos.
3. Construir un programa que permita leer un número entero y que cuente la posición de cada uno de los dígitos que lo componen.
4. Construir un programa que permita leer un entero de 4 dígitos y que sume sus dos primeros dígitos.
5. Construir un programa que permita leer un entero y que sume la mitad de su primer dígito con la mitad de su último dígito.
6. Construir un programa que permita leer un número de 3 dígitos y que determine si su primer dígito es mayor que el tercero.
7. Construir un programa que permita leer un número de tres dígitos. Así mismo, determine si su primer dígito es menor que el segundo y si este es menor que el tercero.

```
1  num = int(input("escriba un numero entero de varios digitos:"  
2    ))  
3  u = num % 10  
4  num1 = num - u  
5  num2 = int(num1/10)  
6  pu = num2 % 10
```

```
7 print(num)
8 print(pu, u)
```

Listing 1.25: Encuentra el penúltimo y último dígito de un número entero en Python.

```
1 n = int(input("Escribe un numero entero con varios digitos:"))
2
3 u = n % 10
4 n1 = n - u
5 n2 = int(n1/10)
6 pu = n2 % 10
7
8 suma = pu + u
9 print(suma)
```

Listing 1.26: Suma de los dos últimos elementos de un número entero en Python basado en el paradigma imperativo.

SECCIÓN 1.3

Clase 3

1.3.1. Creación de listas en diferentes lenguajes

Creación de estructuras que muestran como los datos se presentan en forma de lista, tales como: arreglos, vectores, y listas.

Construcción en Java

Construcción en C++

```
1 #include<iostream>
2 #include<list>
3 #include<string>
4
5 using namespace std;
6
7 list<int> cargarLista() {
8     int a, b;
9     list<int> lista;
10    cout << "Escriba el tama o de lista: " << endl;
```

```
11     cin >> a;
12
13     while(a != 0) {
14         cout << "Escribe el elemento a guardar en la lista: "
15         << endl;
16         cin >> b;
17         cout << endl;
18
19         lista.push_back(b);
20
21         a = a - 1;
22     }
23     return lista;
24 }
25 void imprimirLista(list<int> lista) {
26     for( auto i : lista)
27         cout << i << endl;
28 }
29
30 int main() {
31
32     list<int> lisNum = cargarLista();
33     imprimirLista(lisNum);
34
35     lisNum.sort();
36
37     cout << "Lista ordenada " << endl;
38     imprimirLista(lisNum);
39
40     return 0;
41 }
```

Listing 1.27: Creación de una lista en C++ basado en el paradigma funcional.

Construcción en Python

1.3.2. El paradigma funcional

Esta forma de pensar se basa en presentar el código por medio de funciones. Se puede pasar del paradigma imperativo al paradigma funcional con la creación de funciones. Escribir funciones es un aspecto fundamental de la programación funcional. Las funciones permiten a los programadores encapsular el código dentro de bloques reutilizables que desempeñan tareas específicas [2, Página 5].

Los conceptos que se requieren ver en este paradigma, son: **funciones puras** que cuando son dados los mismos argumentos, dicha función debe devolver los mismos valores; **funciones de primer orden** son las funciones que se tratan como valores básicos y que se pueden asignar a variables, pasar como parámetros a otras funciones, o devolver como resultados; **transparencia referencial** donde cada función puede ser vista como un valor que entonces, puede ser reemplazado sin perder el comportamiento del programa; **funciones impuras**, cuando son dados los mismos valores, regresa valores diferentes, a veces las funciones de orden superior pueden ser funciones impuras; **inmutabilidad** es una propiedad cuando se ha creado una entidad, esta no puede ser modificada. Esto difiere bastante del paradigma imperativo, donde una misma entidad puede variar de estados (modificación o mutación); **funciones de orden superior** son las que reciben funciones como argumentos, o devuelven funciones como resultado o ambas cosas.

```
1 def mult(a, b):
2     return a * b
3
4 def op(f, a, b):
5     return f(a, b)
6
7 print(op(mult, 1, 2))
```

Listing 1.28: Función de orden superior en Python basado en el paradigma funcional.

```
1 def getPrimElem(a, b):
2     def prim(x, y):
3         return x
4
5     return prim(a,b)
6
7 print(getPrimElem(1, 2))
```

Listing 1.29: Otro ejemplo de función de orden superior en Python basado en el paradigma funcional.

```
1 #include<iostream>
2 #include<list>
3 #include<string>
4
5 using namespace std;
6
```

```

7 int main() {
8     int n, num;
9     list<int> lisNum;
10
11     cout << "Escriba el tamaño de lista: " << endl;
12     cin >> n;
13
14     while(n != 0) {
15         cout << "Escribe el elemento a guardar en la lista: "
16         << endl;
17         cin >> num;
18         cout << endl;
19
20         lisNum.push_back(num);
21
22         n = n - 1;
23     }
24
25     for(auto i: lisNum)
26         cout << i << endl;
27
28     lisNum.sort();
29
30     cout << "Lista ordenada " << endl;
31
32     for(auto i: lisNum)
33         cout << i << endl;
34
35     return 0;
36 }

```

Listing 1.30: Creación de una lista en C++ basado en el paradigma imperativo.

El siguiente programa es una modificación del programa anterior, por lo que se tiene el siguiente programa en el paradigma funcional:

```

1 #include<iostream>
2 #include<list>
3 #include<string>
4
5 using namespace std;
6
7 list<int> cargarLista() {
8     int a, b;
9     list<int> lista;

```

```
10  cout << "Escriba el tamaño de lista: " << endl;
11  cin >> a;
12
13  while(a != 0) {
14      cout << "Escribe el elemento a guardar en la lista: "
15      << endl;
16      cin >> b;
17      cout << endl;
18
19      lista.push_back(b);
20
21      a = a - 1;
22  }
23  return lista;
24
25 void imprimirLista(list<int> lista) {
26     for( auto i : lista)
27         cout << i << endl;
28 }
29
30 int main() {
31
32     list<int> lisNum = cargarLista();
33     imprimirLista(lisNum);
34
35     lisNum.sort();
36
37     cout << "Lista ordenada " << endl;
38     imprimirLista(lisNum);
39
40     return 0;
41 }
```

Listing 1.31: Creación de una lista en C++ basado en el paradigma funcional.

Creación de funciones en Python

```
1 def potencia(n, e):
2     if e == 0:
3         return 1
4     elif e == 1:
5         return n
6     else:
```

```

7         return potencia(n, e - 1) * n
8
9 print(potencia(2,4))

```

Listing 1.32: Función de orden en Python basado en el paradigma funcional.

```

1 def copyList(lst):
2     return [element for element in lst]
3
4 a = [1,2,3]
5 b = copyList(a)
6 print(a == b)

```

Listing 1.33: Función de orden en Python basado en el paradigma funcional.

```

1 def copyLiist(lst):
2     if not lst:
3         return []
4     else:
5         return copyList([lst[0]]) + copyList(lst[1:])
6 a = [1,2,3]
7 b = copyList(a)
8 print(a == b)

```

Listing 1.34: Función de orden en Python basado en el paradigma funcional.

Uso de la función map Esta función es para asignar a cada elemento una propiedad.

```

1 def incremento(n):
2     return n + 1
3 resultado = map(incremento, [1,2,3])
4 print([*resultado])

```

Listing 1.35: Función de orden en Python basado en el paradigma funcional.

La función **map()** en C++:

```

1 #include<iostream>
2 #include<map>
3
4 using namespace std;
5
6 int main() {
7     map<int, string> personas;
8     personas[1] = "David";
9     personas.insert({2, "Juan"});
10

```

```
11     for(auto const &[k,v]: personas)
12         cout << k << " " << v << endl;
13 }
```

Listing 1.36: Uso de la función map() en C++ para el paradigma imperativo.

Uso de la función anónima lambda Lambda es una función anónima, aquí se presenta la sintaxis de la función lambda en python.

```
1 resultado = map(lambda x: (x, x + 1), [1,2,3])
2 print(*resultado)
```

Listing 1.37: Función de orden en Python basado en el paradigma funcional.

Aplicación de lambda en C++:

```
1 #include<iostream>
2
3 int main() {
4     auto cuadrado = [](int x) -> int {
5         return x * x;
6     };
7
8     std :: cout << "Cuadrado de 5, es: " << cuadrado(5) <<
9     std :: endl;
10
11     return 0;
12 }
```

Listing 1.38: Uso de la lambda en C++ para el paradigma imperativo.

Uso de la función reduce Esta función aplica una reducción sobre una lista.

```
1 from functools import reduce
2 resultado = reduce(lambda x, y: x + y, [1,2,3,4,5])
3 print(resultado)
```

Listing 1.39: Función reduce en Python basado en el paradigma funcional.

En el caso del lenguaje C++ la función entrega la suma de todos los elementos del contenedor, se muestra el uso de la función en el siguiente programa:

```
1 #include<iostream>
2 #include<vector>
```

```

3 #include<numeric>
4 #include<execution>
5
6 using namespace std;
7
8 int main() {
9     vector<int> v = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
10
11     int suma = reduce(v.begin(), v.end());
12     cout << suma << endl;
13
14     int sumaPar = reduce(execution::par, v.begin(), v.end()
15                          , 0);
16     cout << sumaPar << endl;
17 }

```

Listing 1.40: Aplicación de la función **reduce()** en C++ basado en el paradigma funcional.

La función **accumulate()** también entrega la suma de los elementos de un contenedor, pero la suma la hace secuencialmente.

```

1 #include<iostream>
2 #include<vector>
3 #include<numeric>
4
5 using namespace std;
6
7 int main() {
8     vector<int> num = {1, 2, 3, 4, 5};
9
10    int suma = accumulate(num.begin(), num.end(), 0);
11    cout << suma << endl;
12 }

```

Listing 1.41: Aplicación de la función **accumulate()** en C++ basado en el paradigma funcional.

Uso de la función filter Esta función como su nombre lo indica filtra propiedades.

```

1 resultado = filter(lambda n: n > 0, [-1,2,-3,4,-5])
2 print(list(resultado))

```

Listing 1.42: Función filter en Python basado en el paradigma funcional.

En el caso del lenguaje C++, la forma de usar un filtro, se hace de la siguiente manera:

```
1 #include<iostream>
2 #include<vector>
3 #include<algorithm>
4 #include<iterator>
5
6 using namespace std;
7
8 int main() {
9     vector<int> datos = {13, 5, 8, 9, 12, 4, 7, 10, 6};
10    vector<int> filtrados;
11
12    copy_if(datos.begin(), datos.end(), back_inserter(
13    filtrados), [](int x) {
14        return x > 7;
15    });
16
17    for(int x: filtrados)
18        cout << x << " ";
19    cout << endl;
20
21    auto nuevo = remove_if(datos.begin(), datos.end(), [](int
22    x) {
23        return x < 6;
24    });
25
26    datos.erase(nuevo, datos.end());
27
28    for(int x: datos)
29        cout << x << " ";
30    cout << endl;
31
32    return 0;
33 }
```

Listing 1.43: Forma de aplicar el filtro en C++ para el paradigma imperativo.

Para implementar el filtrado en c++ se usan las funciones **copy_if()**, que lo que realiza, es copiar en otro contenedor los elementos que cumplen con un predicado. La función **remove_if()** quita los elementos del contenedor que cumplen con el predicado.

La función **find_if()** muestra el primer elemento que cumple con el predicado de un contenedor.

Creación de funciones en C++

Se puede pasar del paradigma **imperativo** al paradigma **funcional**, como ejemplo la impresión de la suma de los elementos de un arreglo:

```
1 #include<iostream>
2
3 using namespace std;
4
5 int main() {
6     int num[] = [1,2,3,4,5];
7     int t = 0,
8
9     for(int i = 0; i < 5; i++) {
10         t += num[i];
11     }
12
13     cout << t;
14 }
```

Listing 1.44: Creación de una suma de elementos de un arreglo en C++ basado en el paradigma imperativo.

Este mismo programa se pasa al paradigma funcional de la siguiente manera:

```
1 #include<iostream>
2
3 using namespace std;
4
5 void sumArr(int arr[]) {
6     int to = 0;
7     for(int i = 0; i < 5; i++) {
8         to += arr[i];
9     }
10
11     cout << to << endl;
12 }
13
14 int main(){
15     int num[] = {1,2,3,4,5};
16
17     sumArr(num);
18 }
```

18 }

Listing 1.45: Creación de una suma de elementos de un arreglo usando una función en C++ basado en el paradigma funcional: para un solo caso.

El programa para calcular la suma de todos los elementos de un arreglo de una forma más general:

```

1  #include<iostream>
2
3  using namespace std;
4
5  void sumArr(int arr[], int ta) {
6      int to = 0;
7      for(int i = 0; i < ta; i++) {
8          to += arr[i];
9      }
10
11     cout << to << endl;
12 }
13
14 int main(){
15     int num[] = {1,2,3,4,5};
16     int ta = sizeof(num) / sizeof(num[0]);
17
18     sumArr(num, ta);
19 }
```

Listing 1.46: Creación de una suma de elementos de un arreglo usando una función en C++ basado en el paradigma funcional: para casos en general.

Se presenta el programa que calcula el número de líneas en paradigma imperativo:

```

1  std :: vector<int>
2  contarLineasEnArchivos(const std :: vector<std :: string>&
3      archivos) {
4      std :: vector<int> resultado;
5      char c = 0;
6
7      for(const auto& archivo : archivos) {
8          int lineaCon = 0;
9
10         std :: ifstream in(archivo);
11
12         while(in.get(c)) {
13             if(c == '\n') {
```

```
13         lineaCon++;
14     }
15 }
16 resultado.push_back(lineaCon);
17 }
18 return resultado;
19 }
```

Listing 1.47: Cuenta de líneas en archivos en C++ basado en el paradigma imperativo: para casos de tipo entero.

[3, Página 4].

Creación de funciones en Java

Creación de funciones anónimas o uso de lambdas en C++

Uso de las funciones anónimas o lambdas:

```
1 #include<iostream>
2
3 int main() {
4     auto cuadrado = [](int x) -> int {
5         return x * x;
6     };
7
8     std :: cout << "Cuadrado de 5, es: " << cuadrado(5) <<
9     std :: endl;
10
11     return 0;
12 }
```

Listing 1.48: Creación de una función lambda en C++ basado en el paradigma funcional.

```
1 #include<iostream>
2 #include<vector>
3 #include<algorithm>
4
5 int main() {
6     std::vector<int> numeros = {1, 2, 3, 4, 5};
7
8     std::for_each(numeros.begin(), numeros.end(), [](int n) {
9         std::cout << n << " ";
10     });
11 }
```

```
12     std::cout << std::endl;
13     return 0;
14 }
```

Listing 1.49: Creación de uso de lambda para imprimir en C++ basado en el paradigma funcional: para casos de tipo entero.

```
1  #include<iostream>
2  #include<vector>
3
4  using namespace std;
5
6  void aplicarFuncion(const vector<int>& dato, void(*fun)(int))
7  {
8      for(int valor: dato) {
9          fun(valor);
10     }
11 }
12
13 void imprimirValor(int valor) {
14     cout << valor << " ";
15 }
16
17 int main() {
18     vector<int> num = {1,2,3,4,5,6};
19     aplicarFuncion(num, imprimirValor);
20     cout << endl;
21 }
```

Listing 1.50: Creación de uso de lambda para imprimir en C++ basado en el paradigma funcional: para casos de tipo entero.

```
1  #include<iostream>
2  #include<functional>
3
4  using namespace std;
5
6  functional<int(int)> crearAgregador(int offset) {
7      return [offset](int valor) -> int {
8          return valor + offset;
9      };
10 }
11
12 int main() {
13     auto adDiez = crearAgregador(10);
14     cout << "10 + 5 = " << adDiez(5) << endl;
15 }
```

15 }

Listing 1.51: Creación de uso de lambda para imprimir en C++ basado en el paradigma funcional: para casos de tipo entero.

SECCIÓN 1.4

Clase 4

1.4.1. Uso de la linea de comandos

La linea de comandos en C++:

```
1 #include<iostream>
2
3 using namespace std;
4
5 int main(int argc, char* argv[]) {
6     cout << "Nombre del programa: " << argv[0] << endl;
7     cout << "Numero de argumentos: " << argc << endl;
8
9     for(int i = 1; i < argc; ++i) {
10         cout << "Argumento: " << i << ": " << argv[i] << endl;
11     }
12 }
```

Listing 1.52: Uso de la linea de comandos en C++: basado en el paradigma funcional.

La linea de comandos en Java:

```
1 public class LinCom {
2     public static void main(String[] args) {
3         if(args.length > 0) {
4             System.out.println("Primer argumento: " + args[0]
5 + args[1]);
6         }
7
8         System.out.println();
9
10        for(int i = 0; i < args.length; i++)
11            System.out.println(args[i]);
12    }
13 }
```

12 }

Listing 1.53: Uso de la linea de comandos en Java.

Uso del operador `?`, en java, la sintaxis es: **condición ? valor si es verdadero: valor si es falso;**

```

1 public class OpTernario {
2     public static void main(Sstring[] args) {
3         int i, k;
4
5         i = 10;
6         k = i < 0 ? -i: i;
7         System.out.println("El valor absoluto de ");
8         System.out.println(i + " es " + k);
9
10        i = -10;
11        k = i < 0 ? -i: i;
12        System.out.println("El valor absoluto de ");
13        System.out.println(i + " es " + k);
14    }
15 }
```

Listing 1.54: El operador ternario `?` en java.

El operador ternario en C++:

```

1 #include<iostream>
2
3 using namespace std;
4
5 int main() {
6     int edad;
7
8     cout << "Escribir su edad." << endl;
9     cin >> edad;
10    edad > 17 ? cout << "Eres mayor de edad. " << endl : cout
    << "Eres menor de edad. " << endl;
11 }
```

Listing 1.55: Uso del operador ternario (`?`) en C++: basado en el paradigma funcional.

1.4.2. Aplicación del polimorfismo en paradigma funcional

El polimorfismo tiene dos formas, una de ellas es la **sobrecarga**. Otra de las formas de polimorfismo es la **herencia**. Aunque también se presenta el polimorfismo en la **programación genérica**.

Ejemplo de polimorfismo usando la sobrecarga es el siguiente:

```
1 #include <iostream>
2
3 using namespace std;
4
5 void funcion(int i);
6 void funcion(int i, int j);
7 void funcion(double k);
8
9 int main() {
10     funcion(10);
11     funcion(10, 20);
12     funcion(12.23);
13
14     return 0;
15 }
16
17 void funcion(int i) {
18     cout << "En funcion(int), i es " << i << '\n';
19 }
20
21 void funcion(int i, int j) {
22     cout << "En funcion(int, int), i es " << i;
23     cout << ", j es " << j << '\n';
24 }
25
26 void funcion(double k) {
27     cout << "En funcion(double), k es " << k << '\n';
28 }
```

Listing 1.56: La sobrecarga como forma de polimorfismo en C++ para el paradigma imperativo.

Ejemplo de polimorfismo usando la herencia en C++:

```
1 #include<iostream>
2
3 using namespace std;
4
```

```
5 class B {
6     int a;
7     public:
8     int b;
9     void getAB();
10    int getA();
11    void mostrarA(void);
12 };
13
14 class D : public B {
15     int c;
16     public:
17     void mul(void);
18     void desplegar(void);
19 };
20
21 void B :: getAB(void) {
22     a = 5;
23     b = 10;
24 }
25
26 int B :: getA() {
27     return a;
28 }
29
30 void B :: mostrarA() {
31     cout << a << endl;
32 }
33
34 void D :: mul() {
35     c = b * getA();
36 }
37
38 void D :: desplegar() {
39     cout << getA() << endl;
40     cout << b << endl;
41     cout << c << endl;
42 }
43
44 int main() {
45     D d;
46
47     d.getAB();
48     d.mul();
49     d.mostrarA();
```

```
50     d.desplegar();
51
52     d.b = 5;
53     d.mul();
54     d.desplegar();
55 }
```

Listing 1.57: La herencia como forma de polimorfismo en C++ para el paradigma imperativo.

El polimorfismo en la programación genérica:

```
1  #include <iostream>
2  #include "ListaClaT.h"
3  #include <list>
4  #include <string>
5
6  using namespace std;
7
8  int main() {
9
10     ListaClaT<double> lista;
11
12     lista.agregarNodo(2.5);
13     lista.agregarNodo(7.9);
14     lista.agregarNodo(12.6);
15
16     ListaClaT<double *> :: iterator it = lista.begin();
17     cout << *it << endl;
18
19     //lista.sort();
20
21     lista.insertarNodo(3.3);
22
23     lista.borrarNodo(12.6);
24
25     lista.desplegarLista();
26
27     return 0;
28 }
```

Listing 1.58: El polimorfismo en la programación genérica en C++ para el paradigma imperativo.

El siguiente es un archivo que acompaña al programa **ListaEnClaT.cpp**:

```
1  #ifndef LISTACLAT_H
```

```
2 #define LISTACLAT_H
3
4 using namespace std;
5
6 template <class elemTipo>
7 class ListaClaT {
8     private:
9         //template <class elemTipo>
10        struct ListaNodos {
11            elemTipo inf;
12            struct ListaNodos *sig;
13        };
14        //ListaNodos<elemTipo>* nuevoNodo;
15        //ListaNodos<elemTipo>* a;
16        //int long;
17
18        ListaNodos *cabeza;
19
20    public:
21        ListaClaT() {
22            cabeza = 0;
23        }
24
25        ~ListaClaT();
26
27        void agregarNodo(elemTipo);
28        void insertarNodo(elemTipo);
29        void borrarNodo(elemTipo);
30        void desplegarLista() const;
31 };
32
33
34
35 template <class elemTipo>
36 void ListaClaT<elemTipo> :: agregarNodo(elemTipo num) {
37     ListaNodos *nuevoNodo;
38     ListaNodos *a;
39
40     nuevoNodo = new ListaNodos;
41     nuevoNodo -> inf = num;
42     nuevoNodo -> sig = NULL;
43
44     if(!cabeza)
45         cabeza = nuevoNodo;
46     else{
```

```
47     a = cabeza;
48
49     while(a -> sig)
50         a = a -> sig;
51
52     a -> sig = nuevoNode;
53 }
54 }
55
56 template <class elemTipo>
57 void ListaClaT<elemTipo> :: desplegarLista() const {
58     ListaNodos *a;
59
60     a = cabeza;
61
62     while(a) {
63         cout << a -> inf << endl;
64         a = a -> sig;
65     }
66 }
67
68 template <class elemTipo>
69 void ListaClaT<elemTipo> :: insertarNode(elemTipo num) {
70     ListaNodos * nuevoNode;
71     ListaNodos *a;
72     ListaNodos *previoNode = NULL;
73
74     nuevoNode = new ListaNodos;
75     nuevoNode -> inf = num;
76
77     if(!cabeza) {
78         cabeza = nuevoNode;
79         nuevoNode -> sig = NULL;
80     }
81     else {
82         a = cabeza;
83
84         previoNode = NULL;
85
86         while( a != NULL && a -> inf < num) {
87             previoNode = a;
88             a = a -> sig;
89         }
90
91         if(previoNode == NULL) {
```

```
92         cabeza = nuevoNode;
93         nuevoNode -> sig = a;
94     }
95     else {
96         previoNode -> sig = nuevoNode;
97         nuevoNode -> sig = a;
98     }
99 }
100 }
101
102 template <class elemTipo>
103 void ListaClaT<elemTipo> :: borrarNode(elemTipo num) {
104     ListaNodos *a;
105     ListaNodos *previoNode;
106
107     if(!cabeza) {
108         return;
109     }
110
111     if(cabeza -> inf == num) {
112         a = cabeza -> sig;
113         delete cabeza;
114         cabeza = a;
115     }
116     else {
117         a = cabeza;
118
119         while(a != NULL && a -> inf != num) {
120             previoNode = a;
121             a = a -> sig;
122         }
123
124         if(a) {
125             previoNode -> sig = a -> sig;
126             delete a;
127         }
128     }
129 }
130
131 template <class elemTipo>
132 ListaClaT<elemTipo> :: ~ListaClaT() {
133     ListaNodos *a;
134     ListaNodos *siguienteNode;
135
136     a = cabeza;
```

```
137
138     while(a != NULL) {
139         siguienteNodo = a -> sig;
140
141         delete a;
142
143         a = siguienteNodo;
144     }
145 }
146 #endif
```

Listing 1.59: El polimorfismo en la programación genérica en C++ para el paradigma imperativo.

SECCIÓN 1.5

Actividades para el paradigma funcional

Realizar las siguientes actividades para el paradigma funcional:

1. Construir un programa que permita leer un número entero y que lo guarde en un arreglo. Use el paradigma funcional.
2. Construir un programa que guarde un listado de frutas cítricas donde se aplique el paradigma funcional.
3. Construir un programa que permita leer un número entero y que lo guarde en un arreglo, que entregue el primer y último dígito. Donde se use el paradigma funcional.

SECCIÓN 1.6

El paradigma lógico

CAPÍTULO 2

Paradigma orientado a objetos

Este paradigma consiste en agrupar en clases los objetos. Los lenguajes de programación presentan sintaxis para la representación de las clases y la creación de los objetos, donde también se representan las operaciones con los objetos, dándole las propiedades que representan la forma de utilizar este paradigma.

SECCIÓN 2.1

Creación de clase y objetos en C++

La programación se puede realizar utilizando diferentes paradigmas, uno de ellos es el paradigma orientado a objetos. El lenguaje que se utiliza para programar en este paradigma es el Java, Python, y C++. El que se usa más en el medio académico es el Java, ya que muchos de los estudiantes ya vienen con bases para este lenguaje.

Para el lenguaje C++ la forma de crear las clases y los objetos se presentan en los siguientes programas:

```
1 #include<iostream>
2 #include<cstdlib>
3
4 using namespace std;
5
6 class Rectangulo {
7     double ancho;
8     double largo;
9     public:
10    void setAncho(double);
```

```
11     void setLargo(double);
12     double getAncho() {
13         return ancho;
14     }
15     double getLargo() {
16         return largo;
17     }
18     double getArea() {
19         return ancho * largo;
20     }
21 };
22
23 void Rectangulo :: setAncho(double a) {
24     if(a >= 0)
25         ancho = a;
26     else {
27         cout << "Ancho invalido " << endl;
28         exit(EXIT_FAILURE);
29     }
30 }
31
32 void Rectangulo :: setLargo(double l) {
33     if(l >= 0)
34         largo = l;
35     else {
36         cout << "Largo invalido " << endl;
37         exit(EXIT_FAILURE);
38     }
39 }
40
41 int main() {
42     Rectangulo objeto;
43     double rectAncho;
44     double rectLargo;
45
46     cout << "Escribir el ancho del rectangulo: " << endl;
47     cin >> rectAncho;
48     cout << "Escribe el largo del rectangulo: " << endl;
49     cin >> rectLargo;
50
51     objeto.setAncho(rectAncho);
52     objeto.setLargo(rectLargo);
53
54     cout << "Las medidas del rectangulo. " << endl;
55 }
```

```
56     cout << "El ancho es: " << objeto.getAncho() << endl;
57     cout << "El largo es: " << objeto.getLargo() << endl;
58     cout << "El area del rectangulo es: " << objeto.getArea()
    << endl;
59 }
```

Listing 2.1: Creación de clases y objetos en C++ para el paradigma orientado a objetos.

2.1.1. Creación de clase con arreglos en C++

El uso de arreglos dentro de las clases en C++:

```
1  #include<iostream>
2
3  using namespace std;
4
5  const int tam = 5;
6
7  class ArregloCla {
8      int costo[tam];
9      float precio[tam];
10     int cuenta;
11     public:
12     void contar(void) {
13         cuenta = 0;
14     }
15     void setElem(void);
16     void remover(void);
17     void desplegarSum(void);
18     void desplegarElem(void);
19 };
20
21 void ArregloCla :: setElem(void) {
22     cout << "Escribir el precio del elemento: " << endl;
23     cin >> precio[cuenta];
24
25     cout << "Escribir el costo del elemento: " << endl;
26     cin >> costo[cuenta];
27     cuenta++;
28 }
29
30 void ArregloCla :: remover(void) {
31     int a;
32     cout << "Escribir el codigo del elemeto: " << endl;
```

```
33     cin >> a;
34
35     for(int i = 0; i < cuenta; i++) {
36         if(precio[i] == a)
37             precio[i] = 0;
38     }
39 }
40
41 void ArregloCla :: desplegarSum(void) {
42     float sum = 0;
43     for(int i = 0; i < cuenta; i++)
44         sum = sum + precio[i];
45
46     cout << "El valor total es: " << endl;
47 }
48
49 void ArregloCla :: desplegarElem(void) {
50     cout << "E precio es: " << endl;
51
52     for(int i = 0; i < cuenta; i++) {
53         cout << costo[i] << endl;
54         cout << precio[i] << endl;
55     }
56     cout << endl;
57 }
58
59 int main() {
60     ArregloCla orden;
61     orden.contar();
62     int x;
63 }
```

Listing 2.2: Creación de de arreglos dentro de las clases en C++ para el paradigma orientado a objetos.

2.1.2. Uso de static dentro de las clases en C++

El uso de la palabra clave **static** en C++ se aplica de como se muestra en el siguiente programa:

```
1 #include<iostream>
2
3 using namespace std;
4
5 class Estatica {
```

```
6     static int cuenta;
7     int num;
8     public:
9     void setDato(int a) {
10         num = a;
11         cuenta++;
12     }
13
14     void getCuenta(void) {
15         cout << "la cuenta es: " << endl;
16         cout << cuenta << endl;
17     }
18 };
19
20 int Estatica :: cuenta;
21
22 int main() {
23     Estatica a, b, c;
24
25     a.getCuenta();
26     b.getCuenta();
27     c.getCuenta();
28
29     a.setDato(100);
30     b.setDato(200);
31     c.setDato(300);
32
33     cout << "Despues de leer los datos " << endl;
34
35     a.getCuenta();
36     b.getCuenta();
37     c.getCuenta();
38 }
```

Listing 2.3: Aplicación de static dentro de las clases en C++ para el paradigma orientado a objetos.

SECCIÓN 2.2

Creación y manipulación de objetos

```
1 #include <iostream>
2
```

```
3 using namespace std;
4
5 class Tiempo {
6     int horas;
7     int minutos;
8 public:
9     void setTiempo(int h, int m) {
10         horas = h;
11         minutos = m;
12     }
13     void getTiempo(void) {
14         cout << horas << "Horas y : " << endl;
15         cout << minutos << "minuto " << endl;
16     }
17     void sum(Tiempo, Tiempo),
18 };
19
20 void Tiempo :: sum(Tiempo t1, Tiempo t2) {
21     minutos = t1.minutos + t2.minutos;
22     horas = minutos / 60;
23     minutos = minutos % 60;
24     horas = horas + t1.horas + t2.horas;
25 }
26
27 int min() {
28     Tiempo t1, t2, t3;
29
30     t1.setTiempo(2, 45);
31     t2.setTiempo(3, 30);
32
33     t3.sum(t1, t2);
34
35     cout << "t1 = " << t1.getTiempo() << endl;
36     cout << "t2 = " << t2.getTiempo() << endl;
37     cout << "t3 = " << t3.getTiempo() << endl;
38 }
```

Listing 2.4: Manipulación de objetos con las clases en C++ para el paradigma orientado a objetos.

SECCIÓN 2.3

Manipulación de la herencia

```
1 #include <iostream>
```

```
2
3 using namespace std;
4
5 class Estudiante {
6     protected:
7         string nombre;
8         string boleta;
9         string anioAdmision;
10    public:
11        Estudiante() {
12            nombre = " ";
13            boleta = " ";
14            anioAdmision = 0;
15        }
16        Estudiante(string n, string b, int anio) {
17            setEstudiante(n, b, anio);
18        }
19        void setEstudiante(string n, string b, int a) {
20            nombre = n;
21            boleta = b;
22            anioAdmision = a;
23        }
24        const string getNombre() const {
25            return nombre;
26        }
27        const string getBoleta() const {
28            return boleta;
29        }
30        int getAnioAdmision() const {
31            return anioAdmision;
32        }
33        virtual int getRestoHoras() const = 0;
34    };
```

Listing 2.5: Manipulación de la herencia con las clases en C++ para el paradigma orientado a objetos.

SECCIÓN 2.4

Creación de clases abstractas

Como las clases son una colección de objetos, se les puede dar el uso para crear objetos. Pero las clases abstractas son, para ser utilizadas como clases base:

```
1 #include<iostream>
2
3 using namespace std;
4
5 class Figura {
6     public:
7         virtual void dibujar() = 0;
8         virtual ~Figura() {}
9 };
10
11 class Circulo : public Figura {
12     public:
13         void dibujar() override {
14             cout << "Dibujando un circulo " << endl;
15         }
16 };
17
18 int main() {
19     Circulo c;
20     c.diburar();
21 }
22 }
```

Listing 2.6: Manipulación de la herencia con las clases en C++ para el paradigma orientado a objetos.

[4, página 65].

SECCIÓN 2.5

Datos y métodos privados y protegidos

Los miembros protegidos de una clase base son parecidos a los miembros privado, pero pueden ser accedidos por medio de las clases derivadas. Se acceden por la herencia.

```
1 #include<iostream>
2
3 using namespace std;
4
5 class GradoActividad {
6     protected:
7         double calificacion;
8     public:
9         GradoActividad() {
```

```
10     calificacion = 0.0;
11 }
12 GradoActividad(double c) {
13     calificacion = c;
14 }
15 void setCalificacion(double c) {
16     calificacion = c;
17 }
18 double getCalificacion() const {
19     return calificacion;
20 }
21 char getGradoLetra() const;
22 };
23
24 class ExamenFinal : public GradoActividad {
25     int numPregunta;
26     double puntoPregunta;
27     int numNoContestado;
28 public:
29     ExamenFinal() {
30         numPregunta = 0;
31         puntoPregunta = 0.0;
32         numNoContestado = 0;
33     }
34     ExamenFinal(int pregunta, int noContestada) {
35         calificar(pregunta, noContestada);
36     }
37     void calificar(int, int);
38     void setCalificacion();
39     double getNumPregunta() const {
40         return numPregunta;
41     }
42     double getPuntoPregunta() const {
43         return puntoPregunta;
44     }
45     int getNoContestado() const {
46         return numNoContestado;
47     }
48 };
49
50 void ExamenFinal :: calificar(int, int) {
51     double calificacion;
52
53     numPregunta = pregunta;
54     numNoContestado = noContestada;
```

```

55
56     ponderacionPregunta = 100.0 / numPregunta;
57
58     calificacion = 100.0 - (noContestada *
59     ponderacionPregunta);
60
61     setCalificacion(calificacion);
62     setCalificacion();
63 }
64
65 void ExamenFinal :: setCalificacion() {
66     double fraccion = calificacion - static_cast<int>(
67     calificacion);
68
69     if(fraccion == 0.5) {
70         calificacion += (1.0 - fraccion);
71     }
72 }
73
74 int main() {
75     int pregunta;
76     int noContestada;
77
78     cout << "De cuantas pregntas consta el EF: " << endl;
79     cin >> preguntas;
80
81     cout << "Cuantas preguntas no contesto el Estudiante: "
82     << endl;
83     cin >> noContestada;
84
85     ExamenFinal examen(preguntas , noContestada);
86     cout << setprecision(2) << fixed;
87     cout << "Cada pregunta vale: " << examen.getPonderacion()
88     <<
89     "puntos." << endl;
90     cout << "La calificacion del examen es " << examen.
91     getCalificacion() << endl;
92     cout << "El grado del examen es: " << examen.
93     getGradoLetra() <<
94     endl;
95 }

```

Listing 2.7: Modificadores privados y protegidos con las clases en C++ para el paradigma orientado a objetos.

[5, página 902].

SECCIÓN 2.6

Creación de interfaces y su uso

Una interfaces es una especie de clase abstracta, sin embargo en **Java** esta clase abstracta se representa con **interface**.

```
1 public interface QueComer {
2     public abstract String queCome();
3 }
4
5 class Pollo extends Animal implements QueComer {
6     public String queCome() {
7         return "El pollo come: maiz quebrado ";
8     }
9 }
10
11 public class Comestible {
12     public static void main(String[] args) {
13         Objeto[] os = {new Tigre(), new Pollo(), new Manzana
14     };
15 }
```

Listing 2.8: Creación de una interface en Java.

2.6.1. Aplicación de la interface para crear lamdas en java

```
1 @FunctionalInterface
2 interface Operacion {
3     double calcular(double a, double b);
4 }
5
6 public class Lambda1 {
7     public static void main(String[] args) {
8         Operacion suma = (val1, val2) -> val1 + val2;
9         Operacion multi = (a, b) -> a * b;
10        Operacion divi = (c, d) -> c / d;
11
12        System.out.println("Suma: " + calcularMetodo(suma
13        , 25, 36));
14        System.out.println("Multiplicacion: " +
15        calcularMetodo((a, b) -> a * b, 25, 36));
```

```

14
15         System.out.println("Multiplicacion: " +
16         calcularMetodo(multi,25,36));
17         System.out.println("Division: " + calcularMetodo((c,d
18         ) -> c / d,10,2));
19     }
20
21     private static double calcularMetodo(Operacion o, double
22     a, double b) {
23         return o.calcular(a,b);
24     }
25 }

```

Listing 2.9: Uso de la lambda en Java.

```

1 //package lambda;
2
3 @FunctionalInterface
4 interface DesCal {
5     double calDes(double precio);
6 }
7
8 class Lambda2 {
9     public static void main(String[] args) {
10         double precioP1 = 80.0;
11         double precioP2 = 120.0;
12
13         DesCal descuento10 = precio -> precio * 0.10;
14         DesCal precioPer = precio -> {
15             if(precio > 100) {
16                 return precio * 0.15;
17             }
18             else {
19                 return precio * 0.05;
20             }
21         };
22         System.out.println(descuento10.calDes(precioP1));
23         System.out.println(descuento10.calDes(precioP2));
24         System.out.println(precioPer.calDes(precioP1));
25         System.out.println(precioPer.calDes(precioP2));
26     }
27 }

```

Listing 2.10: Uso de la lambda en Java.

```

1 @FunctionalInterface

```

```

2 interface CalCarA {
3     double calArea(int a,int b);
4 }
5
6 @FunctionalInterface
7 interface CalCarV {
8     double calVol(int a,int b,int c);
9 }
10
11 @FunctionalInterface
12 interface CalCarC {
13     double calAreaC(int a);
14 }
15
16 public class Lambda3 {
17     public static void main(String[] args) {
18         CalCarA aR = (int val1, int val2) -> val1 * val2;
19         CalCarV vR = (int val1, int val2,int val3) -> val1 *
20         val2 * val3;
21         CalCarC aC = (int val1) -> Math.PI * val1 * val1;
22
23         System.out.println(aR.calArea(10,20));
24         System.out.println(vR.calVol(10,20,30));
25         System.out.println(aC.calAreaC(10));
26     }
27 }

```

Listing 2.11: Uso de la lambda en Java.

```

1 import java.util.Arrays;
2 import java.util.List;
3 import java.util.stream.Collectors;
4 import java.util.stream.Stream;
5
6 public class ListasLamb3 {
7     public static void main(String[] args) {
8
9         Stream<String> listaNombresOrdenados = Stream.of("
10         Pedro","Juan","Maria","Ana","Carlos")
11
12         // .sorted();
13         // .map(s -> s.toUpperCase());
14         // .count();   Se le quita Stream<String> a la linea
15         9
16         // y se pone long y se pone en print
17         listaNombresOrdenados

```

```

15      // en lugar listaNueva
16
17      List<String> nuevaLista = listaNombresOrdenados.
collect(Collectors.toList());
18
19      System.out.println(nuevaLista);
20
21  }
22 }
23
24 // funci n anyMatch(se pone una lambda);
25 // startsWith(s mbolo con que comienza);

```

Listing 2.12: Uso de la lambda con el uso de listas en Java.

SECCIÓN 2.7

La GUI en Java para interfaces graficas

La forma de crear la interface grafica se hace de la siguiente manera:

```

1 import javax.swing.JOptionPane;
2
3 public class HolaGUI {
4     public static void main(String args[]) {
5         int a = Integer.parseInt(JOptionPane.showInputDialog(
"Escriba un entero a"));
6         int b = Integer.parseInt(JOptionPane.showInputDialog(
"Escriba otro enetro b"));
7         int suma = a + b;
8         JOptionPane.showMessageDialog(null, "La suma es:" +
suma);
9     }
10 }

```

Listing 2.13: Creación uno de una interface grafica con el uso de GUI en Java.

```

1 import javax.swing.JOptionPane;
2
3 public class ImpPreGUI {
4     public static void main(String args[]) {
5         String nomPro;
6         double pre;
7         int can;
8
9         nomPro = JOptionPane.showInputDialog("Nombre del
producto: ");

```

```

10     pre = Double.parseDouble(JOptionPane.showInputDialog(
11         "Precio del producto: "));
12     can = Integer.parseInt(JOptionPane.showInputDialog("
13         Que cantidad?; "));
14     JOptionPane.showMessageDialog(null, "Producto: " +
15         nomPro + "Cantidad: " + can +
16         "\nEl precio a pagara es: " + pre * can);
17 }

```

Listing 2.14: Creación dos de una interface grafica con el uso de GUI en Java.

```

1  import javax.swing.JOptionPane;
2
3  public class CalculoParametros {
4      public static void main(String args[]) {
5          double radio;
6          double area;
7          double circunferencia;
8
9          String radioCad;
10         String salidaCad;
11
12         radioCad = JOptionPane.showInputDialog("Escriba
13         el radio: ");
14
15         radio = Double.parseDouble(radioCad);
16
17         area = Math.PI * radio * radio;
18         circunferencia = 2 * Math.PI * radio;
19
20         salidaCad = "Radio: " + radio + "\n" +
21         "Area: " + area + " unidades cuadradas \n" +
22         "unidades";
23
24         JOptionPane.showMessageDialog(null, salidaCad, "
25         Circulo",
26         JOptionPane.INFORMATION_MESSAGE);
27
28         System.exit(0);
29     }
30 }

```

Listing 2.15: Creación tres de una interface grafica con el uso de GUI en Java.

```

1  import javax.swing.*;

```

```

2
3 public class Etiqueta extends JFrame {
4     private static final int ANCHO = 350;
5     private static final int ALTURA = 100;
6
7     public Etiqueta() {
8         setTitle("Etiqueta, letras, y colores");
9         setSize(ANCHO, ALTURA);
10        setDefaultCloseOperation(EXIT_ON_CLOSE);
11    }
12
13    public static void main(String args[]) {
14
15        Etiqueta oG = new Etiqueta();
16    }
17 }

```

Listing 2.16: Creación cuatro de una interface grafica con el uso de GUI en Java.

```

1 import javax.swing.JFrame;
2
3 public class Forma {
4     public static void main(String[] args) {
5         JFrame form = new JFrame("Un cuadro");
6         form.setSize(400,300);
7         form.setLocationRelativeTo(null);
8         form.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
9         form.setVisible(true);
10    }
11 }

```

Listing 2.17: Creación cinco de una interface grafica con el uso de GUI en Java.

```

1 import javax.swing.*;
2
3 public class FormaConComponentes {
4     public static void main(String[] args) {
5         JFrame form = new JFrame("Forma con componentes");
6
7         JButton bot = new JButton("Correcto");
8         form.add(bot);
9
10        form.setSize(400,300);
11        form.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

```

```

12         form.setLocationRelativeTo(null);
13         form.setVisible(true);
14     }
15 }

```

Listing 2.18: Creación seis de una interface grafica con el uso de GUI en Java.

```

1  import javax.swing.JLabel;
2  import javax.swing.JTextField;
3  import javax.swing.JFrame;
4  import java.awt.FlowLayout;
5
6  public class MostrarFlujo extends JFrame {
7      public MostrarFlujo() {
8          setLayout(new FlowLayout(FlowLayout.LEFT, 5, 20));
9
10         add(new JLabel("Nombre"));
11         add(new JTextField(8));
12         add(new JLabel("MI"));
13         add(new JTextField(1));
14         add(new JLabel("Apellido"));
15         add(new JTextField(8));
16     }
17
18     public static void main(String[] args) {
19         MostrarFlujo form = new MostrarFlujo();
20         form.setTitle("Mostrar");
21         form.setSize(210, 200);
22         form.setLocationRelativeTo(null);
23         form.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
24         form.setVisible(true);
25     }
26 }

```

Listing 2.19: Creación siete de una interface grafica con el uso de GUI en Java.

2.7.1. Creación de listas para aplicación a un proyecto

El siguiente es un programa para comenzar a crear listas en Java:

```

1  import java.util.List;
2  import java.util.Arrays;
3  import java.util.stream.Stream;
4  import java.util.stream.Collectors;

```

```
5 import java.util.Comparator;
6
7 class Persona {
8     String nombre;
9     int edad;
10
11     public Persona(String nombre,int edad) {
12         this.nombre = nombre;
13         this.edad = edad;
14     }
15
16     public void setNombre(String nombre) {
17         this.nombre = nombre;
18     }
19
20     public String getNombre(){
21         return nombre;
22     }
23
24     public void setEdad(int edad) {
25         this.edad = edad;
26     }
27
28     public int getEdad(){
29         return edad;
30     }
31
32     @Override
33     public String toString() {
34         return "Persona [nombre = " + " " + nombre + ", edad = "
35             + " " + edad + " ]";
36     }
37 }
38
39 public class ListasLamb11 {
40     public static void main(String[] args) {
41         List<Persona> ciudadanos = Arrays.asList(
42             new Persona("Juan",25),
43             new Persona("Maria",30),
44             new Persona("Pedro",40),
45             new Persona("Ana",20),
46             new Persona("Lucas",35),
47             new Persona("Jose",65)
48         );
49     }
50 }
```

```
49     double promedioEdad = ciudadanos.stream()
50         .mapToInt(Persona::getEdad)
51         .average()
52         // .sum()
53         // .min()
54         // .max()
55         // .summaryStatistics()
56         .orElse(0.0);
57     System.out.println("El promedio de edad es:" +
promedioEdad);
58
59     List<Persona> ciudadanosOrdenados = ciudadanos.stream
60     ()
61         .sorted(Comparator.comparing(Persona::getEdad))
62         .collect(Collectors.toList());
63     System.out.println(ciudadanosOrdenados);
64
65     System.out.println();
66     ciudadanosOrdenados.forEach(System.out::println);
67 }
```

Listing 2.20: Creación de una lista en Java.

SECCIÓN 2.8

Los paradigmas de programación

SECCIÓN 2.9

Breve historia de los paradigmas de programación

[6, página 65].

SECCIÓN 2.10

Actividades para el capítulo 2

```
1 interface Comida {
2     void queCome();
```

```
3 }
4
5 class TigreCom implements Comida {
6     public String nomAni;
7     public TigreCom(String nomAni) {
8         this.nomAni = nomAni;
9     }
10
11     String getNom() {
12         return nomAni;
13     }
14
15     public static void main(String[] args) {
16         TigreCom ti = new TigreCom("Tigre");
17         Elefante el = new Elefante();
18         Chimpace chi = new Chimpace();
19
20         System.out.println();
21         System.out.println(ti.getNom()); ti.queCome();
22         el.queCome();
23         chi.queCome();
24     }
25
26     public void queCome() {
27         System.out.println("Come carne");
28     }
29 }
30
31 class Elefante implements Comida {
32     public void queCome() {
33         System.out.println("Come pasto");
34     }
35 }
36
37 class Chimpace implements Comida {
38     public void queCome() {
39         System.out.println("Come frutas");
40     }
41 }
```

Listing 2.21: Creación de una interface en Java.

```
1 abstract class FigGeo {
2     String nom;
3
4     public FigGeo(String nom) {
```

```
5         this.nom = nom;
6     }
7
8     abstract double areaFig();
9 }
10
11 class Rectangulo extends FigGeo {
12     double altura;
13     double ancho;
14
15     public Rectangulo(String nom, double altura, double ancho
16 ) {
17         super(nom);
18         this.altura = altura;
19         this.ancho = ancho;
20     }
21
22     double areaFig() {
23         return altura * ancho;
24     }
25 }
26
27 class CalculoFig {
28     public static void main(String[] args) {
29         Rectangulo r1 = new Rectangulo("PrimRec");
30     }
31 }
```

Listing 2.22: Creación de clases abstractas en Java.

```
1 class Animal:
2     def __init__(self, especie, come):
3         self.especie = especie
4         self.come = come
5
6     def presencia(self):
7         pass
8
9     def caracteristicas(self):
10         print("Soy de la especie", self.especie)
11         print("Y como", self.come)
12
13 class Tigre(Animal):
14     def __init__(self, especie, come):
15         super().__init__(especie, come)
16
```

```
17     def presencia(self):  
18         print("El tigre ruge.")  
19  
20 ti = Tigre("mamifero", "carne")  
21  
22 ti.presencia()  
23 ti.caracteristicas()
```

Listing 2.23: Creación de clases en Python basado en el paradigma funcional.

CAPÍTULO 3

Multiparadigmas

La programación se puede realizar utilizando diferentes paradigmas, uno de ellos es el paradigma orientado a objetos. El lenguaje que se utiliza para programar en este paradigma es el Java, Python, y C++. El que se usa más en el medio académico es el Java, ya que muchos de los estudiantes ya vienen con bases para este lenguaje.

SECCIÓN 3.1

La tienda

Es un negocio que vende productos básicos, como:

Semillas comestibles: arroz, frijol, lentejas, garbanzos.

Productos de limpieza: cloro, aromatizan te, jabón de trastes, jabón de baño, jabón de ropa.

Golosinas: chocolates, pan industrial, galletas, dulces.

SECCIÓN 3.2

La cafetería

El negocio de la venta de café: café de diferentes zonas de México, café de algunos países, los cafés en diferentes formas de preparar.

SECCIÓN 3.3

La orquesta

La orquesta madre es la orquesta sinfónica, puede haber orquesta de banda, grupo de rock, grupo de balada.

SECCIÓN 3.4

Tablero de puntuación

SECCIÓN 3.5

Generación de boletas de calificación

SECCIÓN 3.6

La librería

SECCIÓN 3.7

La libreta de apuntes

SECCIÓN 3.8

El Diario de vida

SECCIÓN 3.9

El álbum familiar

SECCIÓN 3.10

La agenda de direcciones

```
1 public class ProgramaMVC {  
2     public static void main(String args[]) {  
3         Controlador objetoControlador = new Controlador();  
4         objetoControlador.iniciar();  
    }
```

```
5     }  
6 }
```

Listing 3.1: Vista en Java

CAPÍTULO 4

Programación en otros paradigmas

CAPÍTULO 5

Exámenes y cuestionarios

La forma de calificar o evaluar es de alguna manera fría. Pero si requiere saber si el proceso de enseñanza es el adecuado es necesario realizar evaluaciones constantes. Por lo que es necesario presentar un objetivo para de lo que se requiere saber con respecto a la eficacia del aprendizaje de los estudiantes. Por lo que se propone evaluar mediante preguntas y presentar otras estrategia.

SECCIÓN 5.1

Cuestionario de acreditación

1. Implementar un programa que genere una lista en C++.
2. Implementar un programa que genere una lista en Python.
3. Implementar un programa que genere una lista en Java.

Bibliografía

- [1] THEOPHILUS EDET, *Imperative Programming: Explicitly describing computation steps for detailed control and efficiency*, Primera edición, Theophilus, USA, 2025.
- [2] WILLIAM E. CLARK, *C++ functional programming for starters: a practical guide with examples*, Primera edición, NOBTREX, USA, 2024.
- [3] IVAN CUKIC, *Functional programming in C++*, Primera edición, Manning Publication, Shelter Island, NY USA, 2019.
- [4] E. BALAGURUSAMY, *Object Oriented Programming with C++*, Cuarta edición, McGraw-Hill Publishing, New Delhi, 7 West Patel Nagar, 2008.
- [5] TONY GADDIS, *Starting out with C++ From Control Structures through Objects*, Octava edición, Pearson Education, USA, 2015.
- [6] KEITH D. COOPER, LINDA TORCZON *Engineering a Compiler*, Segunda edición, Elsevier, USA, 2012.
- [7] AUTOR, UNO, AUTOR DOS y AUTOR TRES, *Compiladores*, número de edición, Editorial, USA, DC, 2012.